Estratégias de Branch

https://engsoftmoderna.info/artigos/gitflow.html

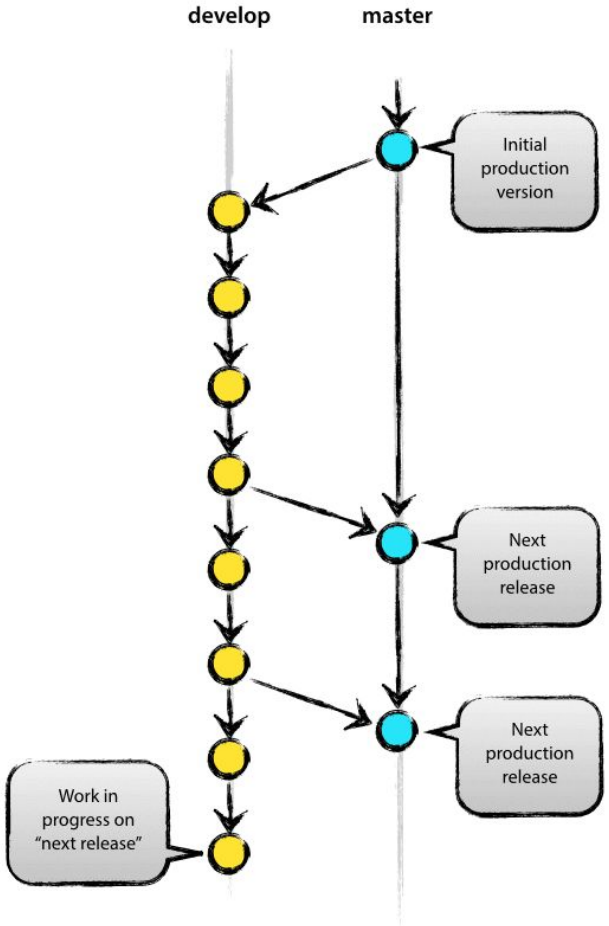
Estratégias de Branch

- Como organizar branches e gerenciar o fluxo de trabalho
- Principais estratégias:
  - Git-flow
  - GitHubFlow
  - Trunk-based Development

Git-Flow

Git-flow

- Estratégia de branch muito comum
- Dois branches permanentes:
  - Master
  - Develop



Fonte: https://nvie.com/posts/a-successful-git-branching-model/

Branches Permanentes

- Master:
  - Código que está pronto para produção
  - Chamado também de main ou trunk
- Develop:
  - Funcionalidades que estão implementadas
  - Mas não passaram por um “teste final”
  - Por exemplo, pela equipe de QA

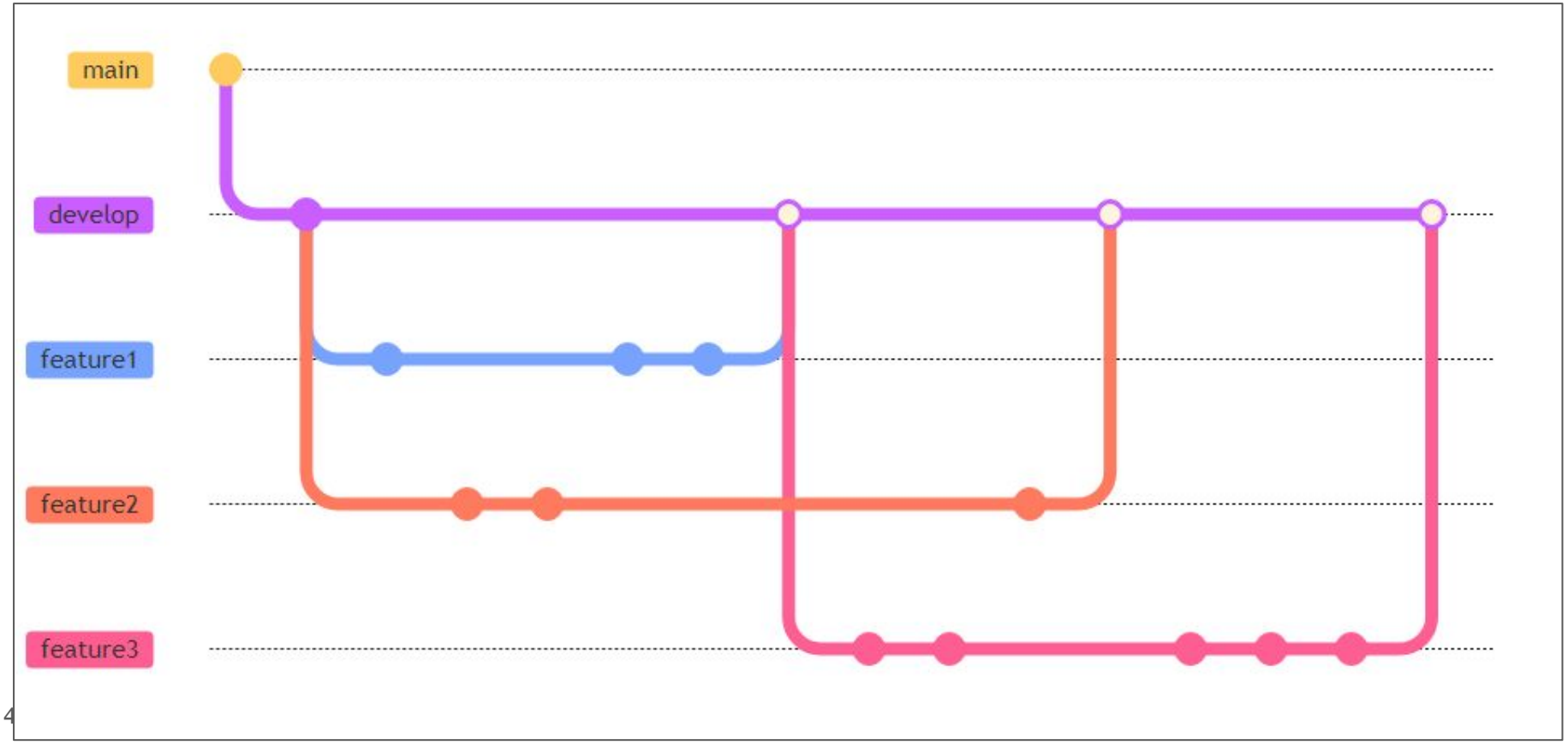
Branches de Apoio

- Três tipos:
  - Feature branches
  - Release branches
  - Hotfix branches
- Temporários (depois de um tempo, são apagados)

Feature Branches

- Branches para implementar uma nova feature
- Nascem de “develop”
- No final, merge de volta em “develop”
- Muitas vezes, existem apenas no repositório local de um desenvolvedor

Feature Branches

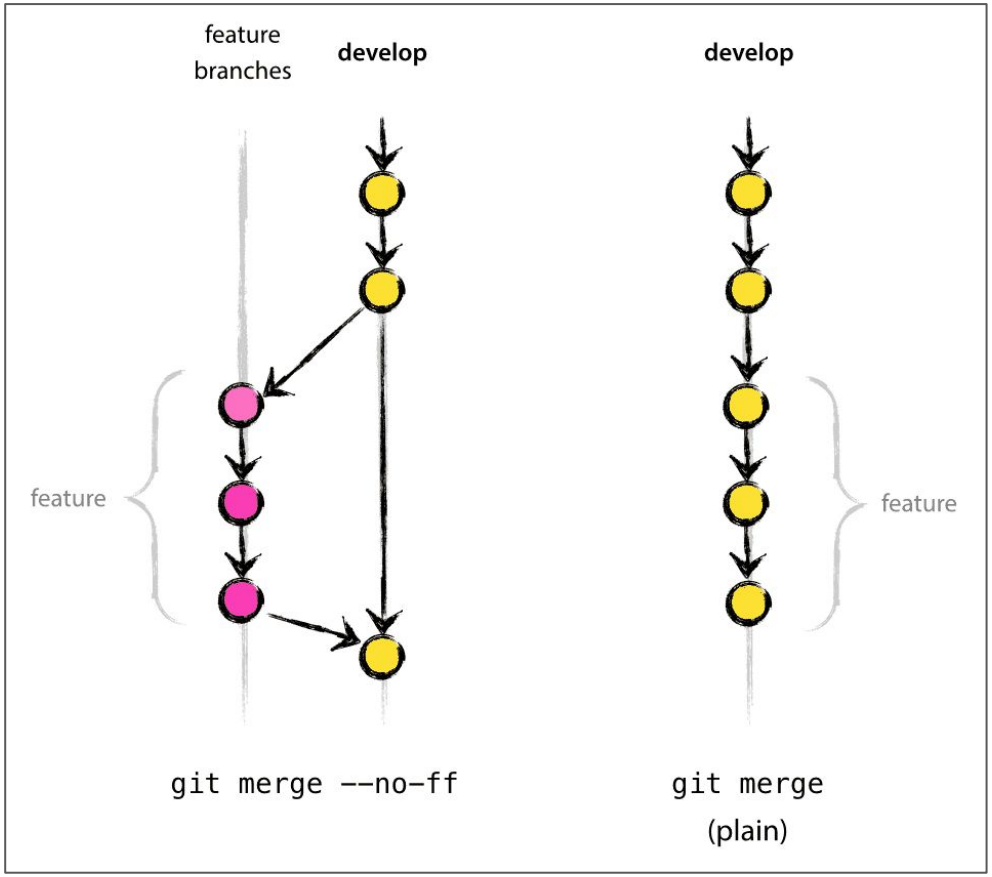


Comandos para criação de feature branches

```
git checkout -b nome-feature develop   % cria feature branch a partir de
develop

[commits da feature]

git checkout develop                   % volta para develop

git merge --no-ff nome-feature         % faz merge de nome-feature em develop
                                       % no-ff: no fast-forwarding (próx.
slide)

git branch -d nome-feature             % remove feature branch

git push origin develop                % push de develop para repo remoto
```

git merge: sem e com fast-forward

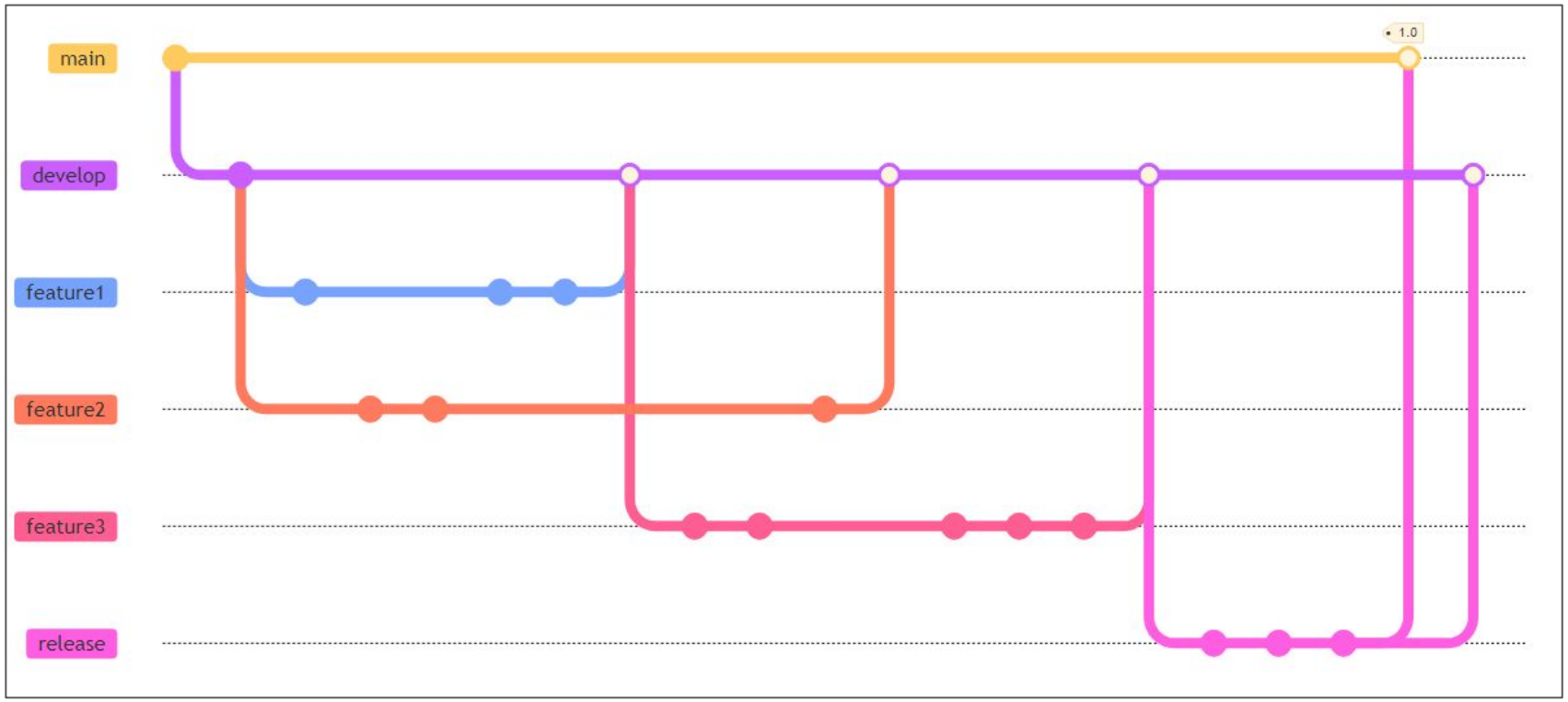
,



Release Branches

- Usados para preparar uma nova release, por exemplo, para ser aprovada por um cliente
- Nascem de develop
- No final:
  - merge no master (com a tag da nova release)
  - merge de volta em develop com bugs corrigidos

Release Branch (último branch da figura)



Comandos para criação de release branches

```
git checkout -b release-1.0 develop   % cria release branch a partir de
develop

[commits da release]

git checkout master                   % move para master
git merge --no-ff release-1.0         % merge do release branch no master
git tag -a 1.0                        % adiciona tag no master

git checkout develop                  % move para develop
git merge --no-ff release-1.0         % merge do release branch em develop

git branch -d release-1.0             % remove release branch

git push origin develop               % push do branch develop
git push origin master                % push do branch master
```

Hotfix Branches

- Branches para corrigir bugs críticos detectado em produção
- Nascem do Master (via tag da release onde bug foi reportado)
- Após correção:
  - merge no master (com nova tag)
  - merge também em develop

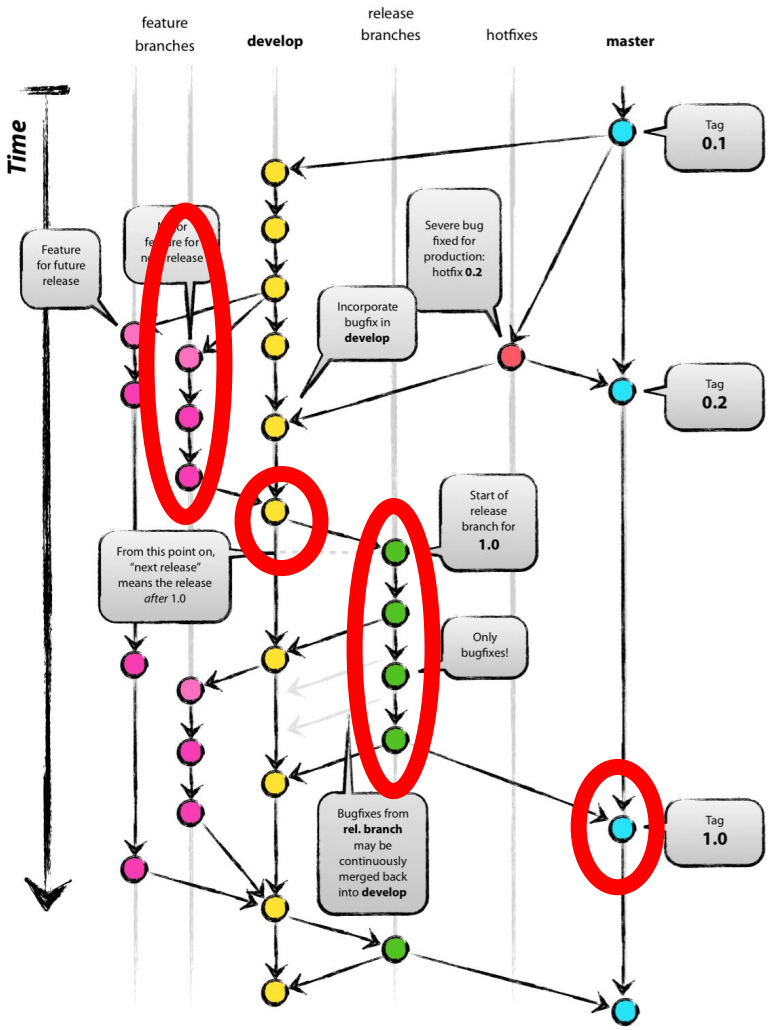
Hotfix Branches (último branch da figura)

Comandos para criação de hotfix branches

```
git checkout -b hotfix-1.2.1 master % cria branch do hotfix a partir do master

[commits do hotfix]

git checkout master                 % muda para o branch master
git merge --no-ff hotfix-1.2.1      % merge to hotfix branch no master
git tag -a 1.2.1                    % adiciona tag no master

git checkout develop                % muda para o branch develop
git merge --no-ff hotfix-1.2.1      % merge do hotfix branch também em develop

git branch -d hotfix-1.2.1          % delta branch do hotfix

git push origin develop             % push de develop para repo remoto
(github)
git push origin master              % push do master para repo remoto (github)
```

Git-flow: Resumo

Feature ⇒ Develop ⇒ Release ⇒
⇒ Master



Git-flow: Uso e desvantagens

- Principal uso quando:
  - Vários clientes com versões diferentes
  - Testes manuais e times de QA
  - Releases precisam de aprovação dos clientes
- Desvantagens:
  - Tendência a ter merges maiores e com mais conflitos
  - E um ciclo de feedback dos clientes mais longo

GitHubFlow

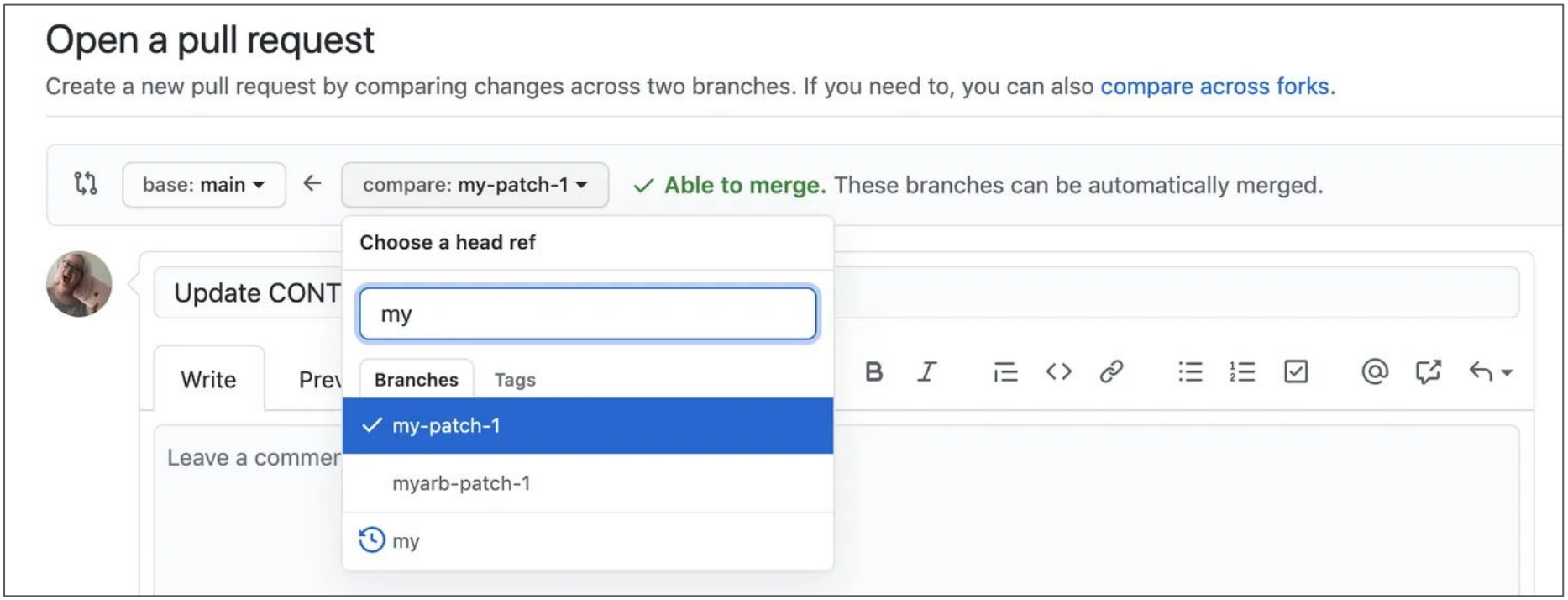
GitHubFlow

- Fluxo comum quando se usa GitHub
- Git-Flow simplificado:
  - Sem branches develop, release e hotfix
  - Apenas feature e master
- Mas com suporte a Pull Requests (PR)

Passos do GitHubFlow

- Desenvolvedor cria “feature branch” no seu repo local
- Implementa uma funcionalidade
- Faz um push do branch para o GitHub
- Entra no GitHub e abre um Pull Request (PR)
  - Pull Request: pedido para alguém revisar seu branch
- Revisor (outro dev) revisa e faz o merge do PR em “main”

Pull Request



GitHubFlow: Uso e desvantagem

- Quando usar:
  - Sistemas com apenas uma versão em produção
  - Exemplo: sistemas Web
- Desvantagem:
  - PRs podem levar muito tempo para serem revisados

Desenvolvimento baseado no Trunk (TBD)

Desenvolvimento baseado no Trunk (TBD)

- Já que merges podem gerar conflitos, TBD defende:
  - Não usar mais branches de desenvolvimento
  - Em vez disso, implementação ocorre diretamente no branch principal
- Branch principal = trunk, master, main